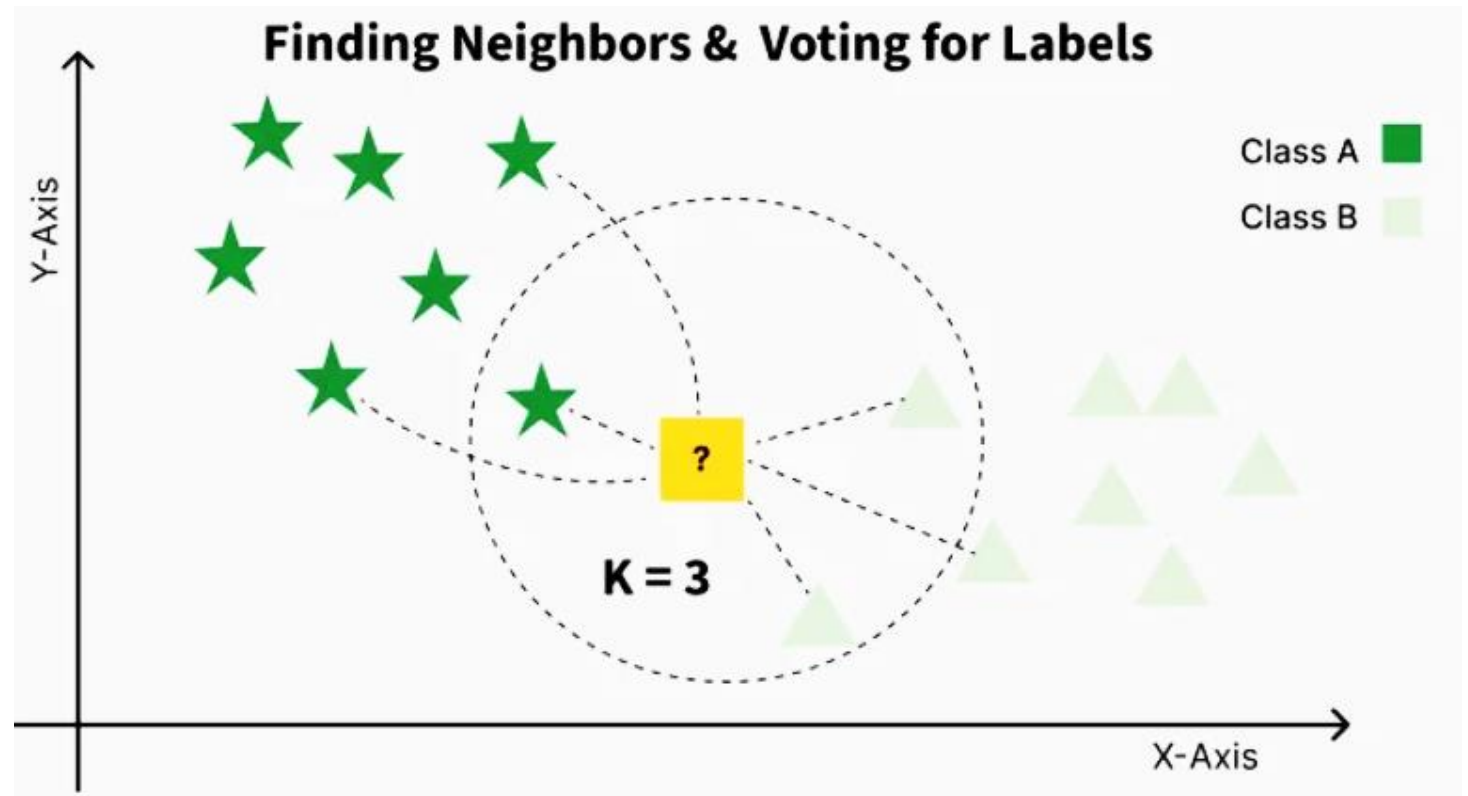


k Nearest Neighbours (KNN)

K-Nearest Neighbors (KNN)

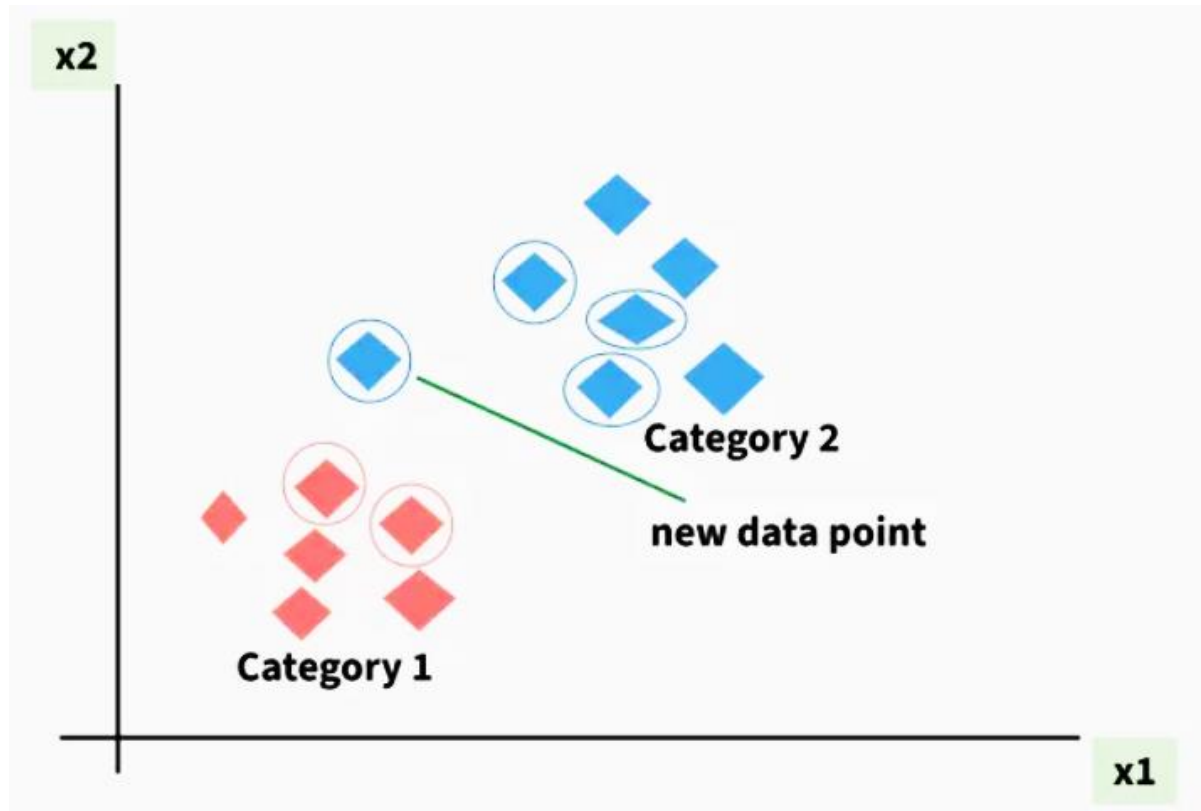
- kNN is a supervised machine learning algorithm generally used for classification but can also be used for regression tasks.
- It works by finding the "**k**" closest data points (neighbors) to a given input and makes a predictions based on the majority class (for classification) or the average value (for regression).



K-Nearest Neighbor Algorithm

- All the instances correspond to points in an n -dimensional feature space.
- Each instance is represented with a set of numerical attributes.
- Each of the training data consists of a set of vectors and a class label associated with each vector.
- Classification is done by comparing feature vectors of different K nearest points.
- Since KNN makes no assumptions about the underlying data distribution it makes it a non-parametric and instance-based learning method.

K-NN is also called as a **lazy learner algorithm** because it does not learn from the training set immediately instead it stores the entire dataset and performs computations only at the time of classification. For example, consider the following table of data points containing two features:



The new point is classified as Category 2 because most of its closest neighbors are blue squares. KNN assigns the category based on the majority of nearby points. The image shows how KNN predicts the category of a new data point based on its closest neighbours.

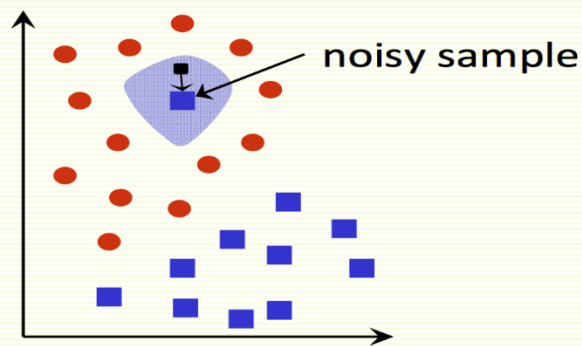
What is 'K' in K Nearest Neighbour?

In the k-Nearest Neighbours algorithm k is just a number that tells the algorithm how many nearby points or neighbors to look at when it makes a decision.

Example: Imagine you're deciding which fruit it is based on its shape and size. You compare it to fruits you already know.

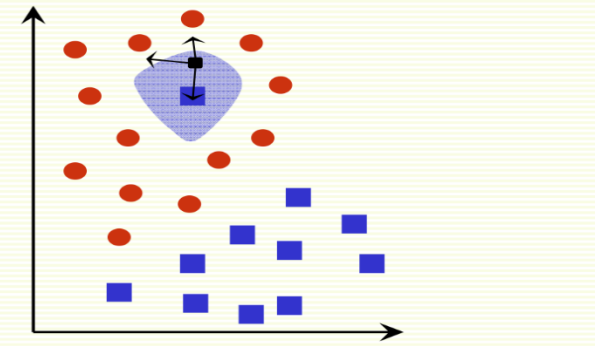
- If $k = 3$, the algorithm looks at the 3 closest fruits to the new one.
- If 2 of those 3 fruits are apples and 1 is a banana, the algorithm says the new fruit is an apple because most of its neighbors are apples.

1 NN

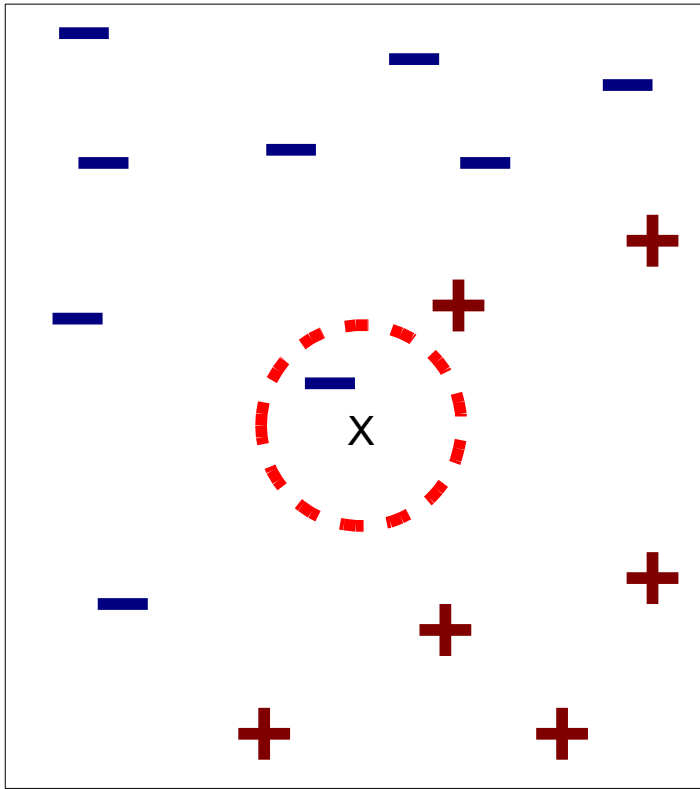


every example in the blue shaded area will be misclassified as the blue class

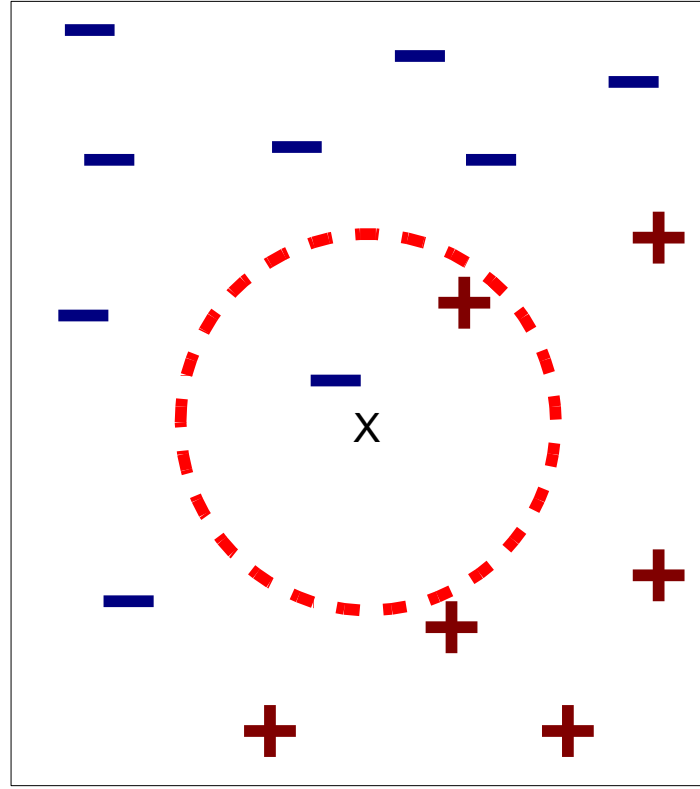
3 NN



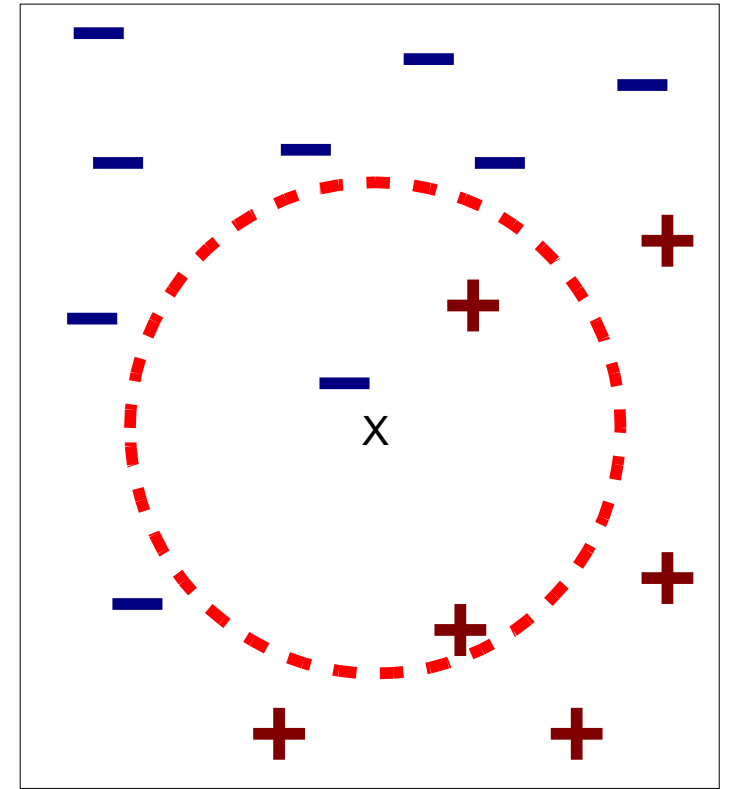
every example in the blue shaded area will be classified correctly as the red class



(a) 1-nearest neighbor



(b) 2-nearest neighbor

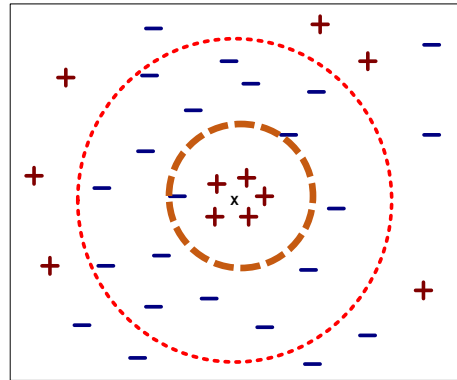


(c) 3-nearest neighbor

K-nearest neighbors of a record x are data points that have the k smallest distance to x

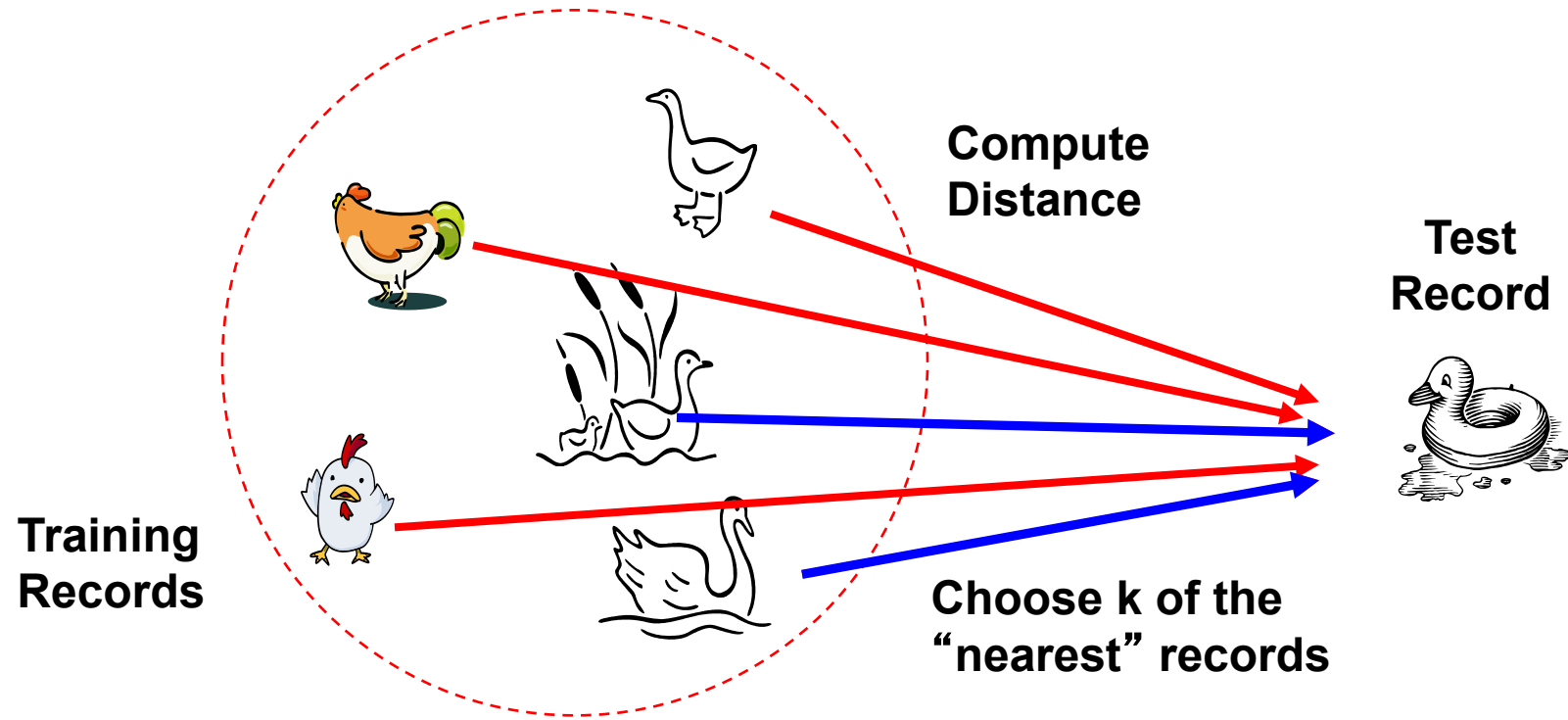
How to choose K?

- If K is **too small**, it is sensitive to noise points.
- Larger K works well. But too large K may include majority points from other classes.



- Rule of thumb is $K < \sqrt{n}$, n is number of examples.

Distance Measure



Distance Metrics Used in KNN Algorithm

KNN uses distance metrics to identify nearest neighbor, these neighbors are used for classification and regression task. To identify nearest neighbor we use below distance metrics:

1. Euclidean Distance

Euclidean distance is defined as the straight-line distance between two points in a plane or space. You can think of it like the shortest path you would walk if you were to go directly from one point to another.

$$\text{distance}(x, X_i) = \sqrt{\sum_{j=1}^d (x_j - X_{i_j})^2}$$

2. Manhattan Distance

This is the total distance you would travel if you could only move along horizontal and vertical lines like a grid or city streets. It's also called "taxicab distance" because a taxi can only drive along the grid-like streets of a city.

$$d(x, y) = \sum_{i=1}^n |x_i - y_i|$$

3. Minkowski Distance

Minkowski distance is like a family of distances, which includes both Euclidean and Manhattan distances as special cases.

$$d(x, y) = \left(\sum_{i=1}^n (x_i - y_i)^p \right)^{\frac{1}{p}}$$

Note: From the formula above, when **p=2**, it becomes the same as the [Euclidean distance formula](#) and when **p=1**, it turns into the [Manhattan distance formula](#). Minkowski distance is essentially a flexible formula that can represent either Euclidean or Manhattan distance depending on the value of p.

3-KNN: Example(1)

Customer	Age	Income	No. credit cards	Class
George	35	35K	3	No
Rachel	22	50K	2	Yes
Steve	63	200K	1	No
Tom	59	170K	1	No
Anne	25	40K	4	Yes
John	37	50K	2	YES

Distance from John

$$\text{sqrt} [(35-37)^2 + (35-50)^2 + (3-2)^2] = 15.16$$

$$\text{sqrt} [(22-37)^2 + (50-50)^2 + (2-2)^2] = 15$$

$$\text{sqrt} [(63-37)^2 + (200-50)^2 + (1-2)^2] = 152.23$$

$$\text{sqrt} [(59-37)^2 + (170-50)^2 + (1-2)^2] = 122$$

$$\text{sqrt} [(25-37)^2 + (40-50)^2 + (4-2)^2] = 15.74$$

KNN Feature Weighting

- Scale each feature by its importance for classification

$$D(a, b) = \sqrt{\sum_k w_k (a_k - b_k)^2}$$

- Can use our prior knowledge about which features are more important
- Can learn the weights w_k using **cross-validation** (to be covered later)

Feature Normalization

- Distance between neighbors could be dominated by some attributes with relatively large numbers.

$$a_i = \frac{v_i - \min v_i}{\max v_i - \min v_i}$$

- Arises when two features are in different scales.
- Important to normalize those features.
 - Mapping values to numbers between 0 – 1.

Applications of KNN

- Recommendation Systems: Suggests items like movies or products by finding users with similar preferences.
- Spam Detection: Identifies spam emails by comparing new emails to known spam and non-spam examples.
- Customer Segmentation: Groups customers by comparing their shopping behavior to others.
- Speech Recognition: Matches spoken words to known patterns to convert them into text.

Advantages of KNN

- **Simple to use:** Easy to understand and implement.
- **No training step:** No need to train as it just stores the data and uses it during prediction.
- **Few parameters:** Only needs to set the number of neighbors (k) and a distance method.
- **Versatile:** Works for both classification and regression problems.

Disadvantages of KNN

- **Slow with large data:** Needs to compare every point during prediction.
- **Struggles with many features:** Accuracy drops when data has too many features.
- **Can Overfit:** It can overfit especially when the data is high-dimensional or not clean.

Problem 1: Unweighted KNN Classification

We have training data with 2 features (X, Y) and two classes (A, B).

Point	Coordinates (X,Y)	Class
P1	(1, 2)	A
P2	(2, 3)	A
P3	(3, 3)	B
P4	(6, 5)	B
P5	(7, 7)	B

Query point: $Q = (3,2)$.

Use $k=3$, unweighted voting.

Step 1: Distances (Euclidean)

- $d(Q,P1) = \sqrt{((3-1)^2+(2-2)^2)} = \sqrt{(4+0)} = 2.00$
- $d(Q,P2) = \sqrt{((3-2)^2+(2-3)^2)} = \sqrt{(1+1)} = 1.41$
- $d(Q,P3) = \sqrt{((3-3)^2+(2-3)^2)} = \sqrt{(0+1)} = 1.00$
- $d(Q,P4) = \sqrt{((3-6)^2+(2-5)^2)} = \sqrt{(9+9)} = 4.24$
- $d(Q,P5) = \sqrt{((3-7)^2+(2-7)^2)} = \sqrt{(16+25)} = 6.40$

Step 2: Pick nearest k=3 neighbors

Nearest are:

- P3 (B, 1.00)
- P2 (A, 1.41)
- P1 (A, 2.00)

Step 3: Majority vote

- Class A = 2 votes
- Class B = 1 vote

Problem 2

Training data (2 features: x, y):

- Class A: A1=(2,4), A2=(4,7), A3=(5,9), A4=(3,6)
- Class B: B1=(7,5), B2=(6,4), B3=(8,6), B4=(6,8)

Query point: q = (5,6)

We'll use Euclidean distance $d(p, q) = \sqrt{(x_p - x_q)^2 + (y_p - y_q)^2}$.

(Compute squared distances first to keep it tidy; I also give the square roots.)

Compute nearest points for k=3 and k=5??

Distances to q:

- A1: $(5 - 2)^2 + (6 - 4)^2 = 3^2 + 2^2 = 9 + 4 = 13 \rightarrow d = \sqrt{13} \approx 3.606$
- A2: $(5 - 4)^2 + (6 - 7)^2 = 1^2 + (-1)^2 = 1 + 1 = 2 \rightarrow d = \sqrt{2} \approx 1.414$
- A3: $(5 - 5)^2 + (6 - 9)^2 = 0^2 + (-3)^2 = 0 + 9 = 9 \rightarrow d = 3$
- A4: $(5 - 3)^2 + (6 - 6)^2 = 2^2 + 0^2 = 4 + 0 = 4 \rightarrow d = 2$
- B1: $(5 - 7)^2 + (6 - 5)^2 = (-2)^2 + 1^2 = 4 + 1 = 5 \rightarrow d = \sqrt{5} \approx 2.236$
- B2: $(5 - 6)^2 + (6 - 4)^2 = (-1)^2 + 2^2 = 1 + 4 = 5 \rightarrow d \approx 2.236$
- B3: $(5 - 8)^2 + (6 - 6)^2 = (-3)^2 + 0^2 = 9 + 0 = 9 \rightarrow d = 3$
- B4: $(5 - 6)^2 + (6 - 8)^2 = (-1)^2 + (-2)^2 = 1 + 4 = 5 \rightarrow d \approx 2.236$

Problem 3

Training set (2 features: X1, X2):

- A1 = (1, 1), class A
- A2 = (2, 2), class A
- B1 = (4, 1), class B
- B2 = (5, 2), class B

Query: q = (3, 2)

Use Euclidean distance. Compare **unweighted** (standard) vs **feature-weighted** with weights $w_1 = 3$ for X1 and $w_2 = 1$ for X2.

(Weighted Euclidean: $d = \sqrt{w_1(x_1 - x_{1,q})^2 + w_2(x_2 - x_{2,q})^2}$.)

Compute weighted squared terms:

1. $d_w(q, A1): 3 * (3 - 1)^2 + 1 * (2 - 1)^2 = 3 * 4 + 1 * 1 = 12 + 1 = 13.$

$$d_w = \sqrt{13} \approx 3.606.$$

2. $d_w(q, A2): 3 * (1)^2 + 1 * (0)^2 = 3 * 1 + 0 = 3.$

$$d_w = \sqrt{3} \approx 1.732.$$

3. $d_w(q, B1): 3 * (-1)^2 + 1 * (1)^2 = 3 * 1 + 1 * 1 = 3 + 1 = 4.$

$$d_w = \sqrt{4} = 2.000.$$

4. $d_w(q, B2): 3 * (-2)^2 + 1 * (0)^2 = 3 * 4 + 0 = 12.$

$$d_w = \sqrt{12} \approx 3.464.$$

Nearest 3 (weighted): A2 (1.732, A), B1 (2.000, B), B2 (3.464, B) $\rightarrow$ votes A:1, B:2 $\Rightarrow$ Prediction = B.

Takeaway: weighting changed distances (A2 moved a bit farther relative to others), but the $k=3$ majority stayed **B** here.

THANK YOU !!